SIEM Log Monitoring & Threat Detection

Detecting Windows failed-logon activity (Event ID 4625) with Wazuh SIEM

Author: Jeetu Rajput

GitHub repository: https://github.com/Charlie9548/SOC-Analyst-L1-Portfolio.git

Project type: SOC Analyst L1 — practical lab

Platform: Wazuh SIEM (Manager on Ubuntu Linux)

Monitored endpoint: Windows 10 Pro, agent name *mylaplab*

Evidence date: 28 October 2026

Executive Summary

This lab shows the first stage of a SOC authentication-monitoring workflow. A failed Windows sign-in was generated on a monitored endpoint, confirmed in the native Windows Security log with PowerShell, and then found as alerts in the Wazuh dashboard. The Wazuh Threat Hunting search returned **6 matching alerts** for Windows Event ID 4625 on agent *mylaplab*, with the rule description *Logon Failure - Unknown user or bad password* at rule level 5.

What this proves: the telemetry pipeline works end to end (endpoint, agent, manager, indexer, dashboard), and I can search and read the resulting alerts.

> **Scope note**
> Six failed-logon events are not, by themselves, proof of a brute-force attack. Source IP, target account, logon type and failure reason must be reviewed before any incident determination. Those fields have not yet been documented, and this report does not claim an attack.

Skills demonstrated

- Verifying a SIEM agent service and endpoint enrolment on Windows.
- Querying the Windows Security log with PowerShell (`Get-WinEvent`).
- Searching and reading alerts in Wazuh Threat Hunting.
- Separating observed evidence from assumptions, which is a core SOC habit.

1. Objective and Scope

The goal was to understand how a SIEM collects and presents endpoint security events, and how a SOC analyst starts an authentication-failure investigation.

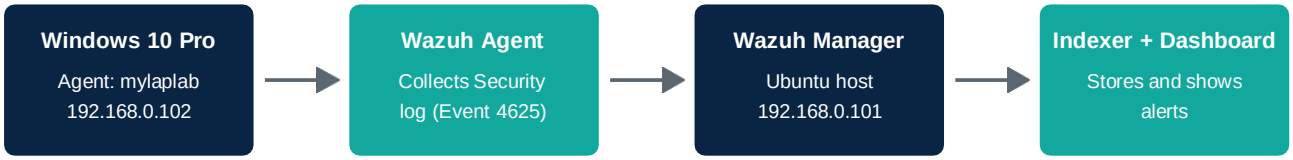
In scope

- Verify that the Windows 10 Wazuh Agent service is running.
- Verify Windows Security Event ID 4625 using PowerShell.
- Search Wazuh Threat Hunting for Event ID 4625 and review the alert description, timestamps and rule level.

Out of scope (not yet completed)

- A custom correlation rule for repeated failures.
- A confirmed brute-force classification.
- Full review of source IP, target username and failure reason for every event.
- A documented response or remediation action.

2. Lab Environment and Architecture



Analyst then runs a Threat Hunting search in the Dashboard: data.win.system.eventID: "4625"

Figure 1: Data flow from the Windows endpoint to the Wazuh dashboard.

Wazuh Manager / dashboard host: Ubuntu Linux, manager IP shown in lab: 192.168.0.101

Monitored endpoint: Windows 10 Pro, agent name mylaplab, dashboard IP shown: 192.168.0.102

Search used in Wazuh: `data.win.system.eventID: "4625"`

Verification tool: PowerShell, `Get-WinEvent` on the Security log, Event ID 4625

3. Background: Windows Event ID 4625

Definition: Event ID 4625 is written to the Windows Security log each time an account fails to log on. The message text is "An account failed to log on." It is one of the most common starting points for authentication investigations.

Fields an analyst reads first

- **TargetUserName** — the account that was tried. A non-existent name suggests guessing; a real admin name raises priority.
- **IpAddress / WorkstationName** — where the attempt came from. One source with many failures is a stronger signal than many sources with one failure each.
- **LogonType** — how the logon was attempted. Type 2 is interactive (keyboard), 3 is network (for example SMB), 7 is unlock, and 10 is RemoteInteractive (RDP).
- **Status / SubStatus** — the reason. For example 0xC000006D is a generic logon failure, 0xC000006A is a correct user with a wrong password, and 0xC0000064 is a user name that does not exist.

Why this matters: a failed logon at the keyboard (type 2) with a mistyped password is usually harmless. Repeated type 3 or type 10 failures from an external address deserve a closer look.

4. Methodology

Step 1 — Check the agent service

PowerShell was used to confirm the Wazuh agent service on the Windows endpoint:

```
Get-Service WazuhSvc
```

Result: the service was in the **Running** state and the dashboard listed the endpoint as *mylaplab*. **OBSERVED**

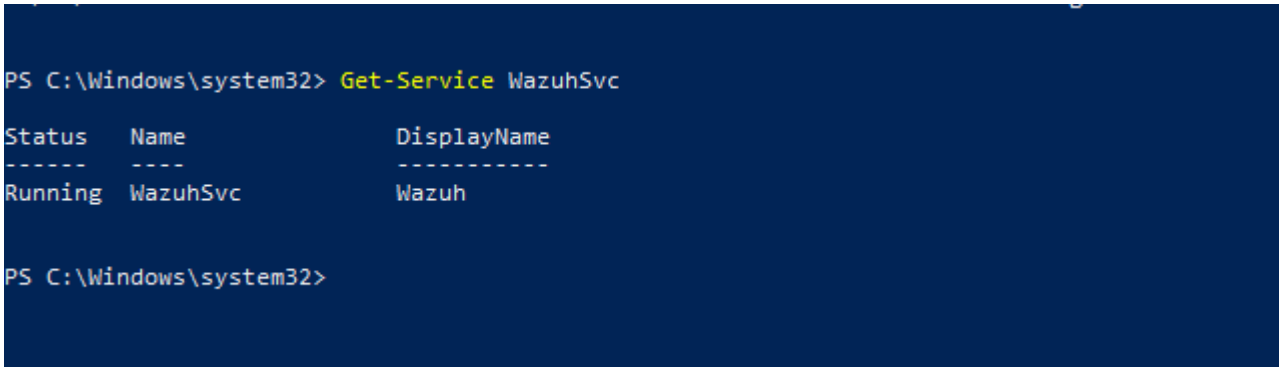


Figure 2: Wazuh agent service status and dashboard agent list.

Step 2 — Generate and verify failed-logon events

A failed sign-in was generated on the lab Windows endpoint, then the Security log was inspected:

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 5
```

Result: the command returned Windows Security-Auditing events with Event ID 4625 and the message "An account failed to log on." **OBSERVED**

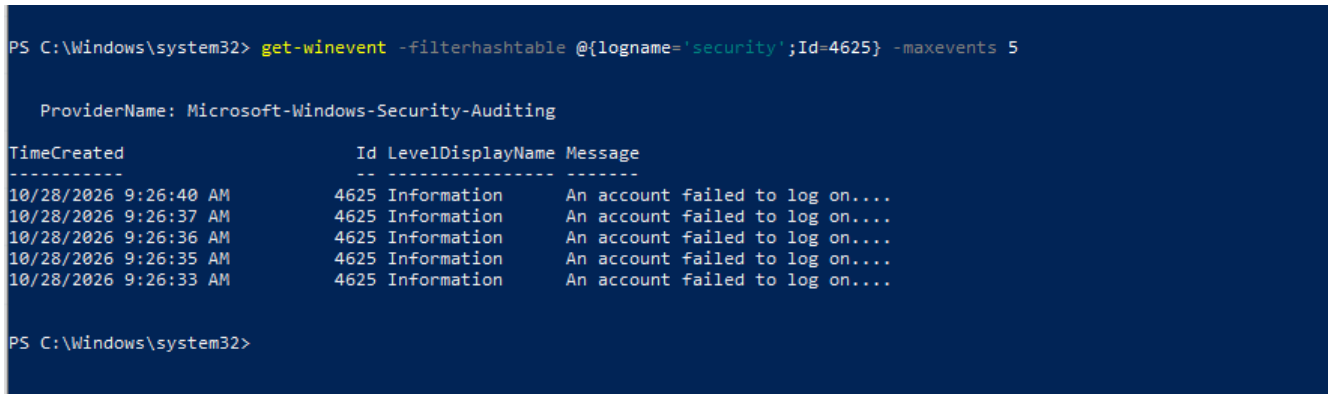


Figure 3: PowerShell output showing Event ID 4625.

Step 3 — Find the alerts in Wazuh

In **Threat Hunting > Events**, the following search was used:

```
data.win.system.eventID: "4625"
```

Result: 6 hits were shown in the Events table. **OBSERVED**

5. Findings

Matching events: 6 hits in the Wazuh Events table

Agent: mylaplab (Windows 10 Pro)

Rule description: Logon Failure - Unknown user or bad password

Rule level: 5

Event identifier: Windows Event ID 4625

Time window: 27 Oct 2026 to 28 Oct 2026 (range displayed in the screenshot)

If the pattern were confirmed as repeated guessing, it would map to **T1110 Brute Force**, for example T1110.001 (Password Guessing) or T1110.003 (Password Spraying). This is a candidate mapping for reference, not a finding from this lab.

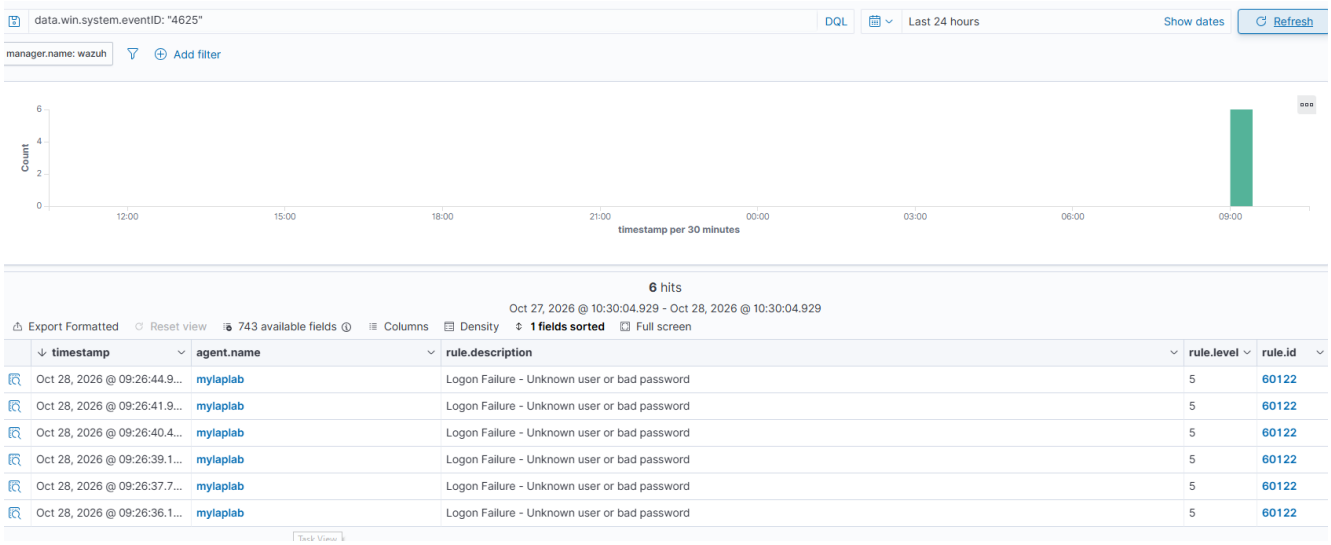


Figure 4: Wazuh Threat Hunting results for Event ID 4625 (6 hits).

Interpretation

The alert was assigned Wazuh rule level 5 (low severity). This indicates a security event that warrants review in context but does not, by itself, confirm a security incident. The analyst should investigate the target account, source IP (if available), logon type, failure status, and surrounding authentication activity before determining whether the activity is suspicious.

> **Analyst caution**
> Do not label this as brute force yet. Inspect target username, source IP, logon type, failure status and substatus, timing, and any successful logon that follows the failures.

6. Triage Approach (how I would investigate next)

This section is a method, not a result. It shows how I would turn the six alerts into a decision.

Questions to answer

- Are all six failures for the same target account, or for different accounts?
- Do they share one source IP, or come from several?
- What logon type is it (keyboard, network, RDP)?
- Are the failures clustered in seconds or minutes, or spread out over hours?
- Is there a successful logon (Event ID 4624) right after the failures?

Leans benign

- One account, keyboard logon, a few failures, then success shortly after.
- Source is the local machine or a known admin workstation.
- Timing matches a user working hours.

Leans suspicious

- Many failures in a short window from one external or unknown source.
- Many different usernames from one source (a sign of password spraying).
- Failures followed by a success on the same account.
- Network or RDP logon types, or activity outside normal hours.

MITRE ATT&CK mapping (candidate only)

Optional: summarise 4625 events by account and source in PowerShell

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 50 |
ForEach-Object {
  $d = ([xml]$_.ToXml()).Event.EventData.Data
  [pscustomobject]@{
    Time      = $_.TimeCreated
    User      = ($d | ? Name -eq 'TargetUserName').'#text'
    SourceIP  = ($d | ? Name -eq 'IpAddress').'#text'
    LogonType = ($d | ? Name -eq 'LogonType').'#text'
    SubStatus = ($d | ? Name -eq 'SubStatus').'#text'
  }
} | Format-Table -AutoSize
```

7. Planned Detection Improvement (not yet implemented)

A single failed logon is normal noise. A SIEM becomes useful when it alerts on a **pattern**. The draft below is a Wazuh frequency rule that would raise severity when the same source produces several failures in a short time. It is an example to be tested in the lab, and it has not been run or validated.

```
<group name="windows,authentication_failures,">
  <rule id="100100" level="10" frequency="5" timeframe="120">
    <if_matched_sid>60122</if_matched_sid>
    <same_field>win.eventdata.ipAddress</same_field>
    <description>Repeated Windows logon failures from same source (lab rule)</description>
    <mitre><id>T1110</id></mitre>
  </rule>
</group>
```

- Place custom rules in `/var/ossec/etc/rules/local_rules.xml` on the manager and restart the manager after editing.
- Confirm the base rule ID (60122 is the usual ID for the unknown-user/bad-password rule) by opening the alert in your own dashboard.
- Test with `/var/ossec/bin/wazuh-logtest` using a sample 4625 event before relying on it.
- Document false positives: users who mistype passwords, service accounts with expired credentials, and scheduled tasks using old passwords.

8. Evidence Checklist

- Windows Wazuh Agent service running — **OBSERVED**
- Windows Security Event ID 4625 in PowerShell — **OBSERVED**
- Six matching Wazuh alerts — **OBSERVED**
- Expanded alert with user and source details — **NOT YET DOCUMENTED**
- Custom repeated-failure detection rule — **NOT IMPLEMENTED**
- Documented response or remediation action — **NOT IMPLEMENTED**

9. Next Steps

Phase 1 — Complete the investigation. Expand one alert and record target username, source IP, logon type and failure reason. Compare timestamps with successful logons.

Phase 2 — Build detection. Implement and test the repeated-failure rule in a controlled lab, then record how many alerts it produces and any false positives.

Phase 3 — Add response. Write a short playbook: verify the source, check the account, reset credentials if needed, and decide whether to block or escalate.

Phase 4 — Publish. Add redacted screenshots and the final README to GitHub.

10. Conclusion

This project gave me practical experience with Wazuh agent monitoring, Windows Security logs, Event ID 4625, and the first steps of a SIEM alert investigation. It is a foundation for authentication monitoring and can be extended into repeated-failure analysis and detection engineering.